

## Source Code Of the **SMART THREAT DETECTION SYSTEM USING AI-BEHAVIORAL ANALYSIS** Project

```
import streamlit as st

import pandas as pd

import numpy as np

import plotly.express as px

import matplotlib.pyplot as plt

import plotly.graph_objects as go

import joblib

import json

import time

import random

import os

from streamlit_autorefresh import st_autorefresh


try:

    from tensorflow.keras.models import load_model

except Exception:

    load_model = None


st.set_page_config(page_title="Smart Threat Detection System", layout="wide")


# ===== GLOBAL CSS =====

st.markdown("""

<style>

@import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;600;700;800&display=swap');

html, body, [class*="css"] {

    font-family: 'Inter', sans-serif;

    background-color: #0B1220;

    color: #E6F1FF;

}


```

```
.auth-box {  
    width: 100%;  
    max-width: 420px;  
    background: #111827;  
    border-radius: 14px;  
    padding: 30px;  
    box-shadow: 0 0 40px rgba(0,0,0,0.6);  
}
```

```
.auth-header {  
    background: linear-gradient(90deg, #7C5CFC, #6D28D9);  
    padding: 14px;  
    text-align: center;  
    border-radius: 10px;  
    font-size: 18px;  
    font-weight: 700;  
    margin-bottom: 20px;  
}
```

```
.auth-footer {  
    text-align: center;  
    margin-top: 15px;  
    color: #9CA3AF;  
}
```

```
.auth-footer span {  
    color: #8B5CF6;  
    font-weight: 600;  
    cursor: pointer;  
}
```

```
.stTextInput input {
    background-color: #020617 !important;
    border-radius: 8px;
    border: 1px solid #1F2937;
    color: white;
}
```

```
.stButton>button {
    width: 100%;
    background: linear-gradient(90deg, #7C5CFC, #6D28D9);
    border-radius: 8px;
    font-weight: 700;
    height: 45px;
}
```

</style>

""", unsafe\_allow\_html=True)

# ===== USER STORAGE =====

USERS\_FILE = "users.json"

def load\_users():

if os.path.exists(USERS\_FILE):

with open(USERS\_FILE, "r") as f:

return json.load(f)

return {}

def save\_users(users):

with open(USERS\_FILE, "w") as f:

json.dump(users, f, indent=4)

```
# ===== SESSION =====
```

```
if "users" not in st.session_state:
```

```
    st.session_state["users"] = load_users()
```

```
if "logged_in" not in st.session_state:
```

```
    st.session_state["logged_in"] = False
```

```
if "register_mode" not in st.session_state:
```

```
    st.session_state["register_mode"] = False
```

```
if "current_user" not in st.session_state:
```

```
    st.session_state["current_user"] = None
```

```
# ===== LOGIN PAGE =====
```

```
def login_page():
```

```
    st.markdown("<br><br>", unsafe_allow_html=True)
```

```
    col1, col2, col3 = st.columns([1, 1.2, 1])
```

```
    with col2:
```

```
        st.markdown("""
```

```
        <div class="auth-box">
```

```
            <div class="auth-header">  Login to AI-Powered Cyber Threat Detection</div>
```

```
        """, unsafe_allow_html=True)
```

```
        u = st.text_input("Username")
```

```
        p = st.text_input("Password", type="password")
```

```
    if st.button("  Login"):
```

```
        if u in st.session_state["users"] and st.session_state["users"][u] == p:
```

```
            st.session_state.logged_in = True
```

```
            st.session_state.current_user = u
```

```
            st.rerun()
```

```
        else:
```

```
            st.error("Invalid username or password")
```

```

if st.button("Register here"):
    st.session_state.register_mode = True
    st.rerun()

# ===== REGISTER PAGE =====

def register_page():
    st.markdown("<br><br>", unsafe_allow_html=True)

    col1, col2, col3 = st.columns([1, 1.2, 1])

    with col2:
        st.markdown("""
        <div class="auth-box">
            <div class="auth-header">
                 Register New Account
            </div>
        """, unsafe_allow_html=True)

        u = st.text_input("Username", placeholder="Username (min 4 characters)")
        fn = st.text_input("First Name")
        ln = st.text_input("Last Name")
        email = st.text_input("Email Address", placeholder="Enter your email")
        p = st.text_input("Password", type="password", placeholder="Create a password")
        cp = st.text_input("Confirm Password", type="password", placeholder="Confirm password")

        if st.button(" + Register"):
            users = st.session_state["users"]
            if u == "" or p == "":
                st.error("All fields are required )
            elif len(u) < 4:

```

```

        st.error("Username must be at least 4 characters ❌")
    elif len(p) < 8:
        st.error("Password must be at least 8 characters ❌")
    elif p != cp:
        st.error("Passwords do not match ❌")
    elif u in users:
        st.error("Username already exists ❌")
    else:
        users[u] = p
        save_users(users)
        st.session_state["users"] = users
        st.success("Account created successfully ✅")
        st.session_state["register_mode"] = False
        st.rerun()

```

```

st.markdown("""
<div class="auth-footer">
    Already have an account? <span>Login here</span>
</div>
</div>
""", unsafe_allow_html=True)

```

```

if st.button("Login here"):
    st.session_state["register_mode"] = False
    st.rerun()

```

# ===== DASHBOARD (UNCHANGED UI) =====

```
def dashboard_page():
```

```

    st_autorefresh(interval=5000, key="dashboard_refresh")

```

```
if "tick" not in st.session_state:
```

```
    st.session_state.tick = 0
```

```
st.session_state.tick += 1
```

```
t = st.session_state.tick
```

```
ATTACK_TYPES = [
```

```
    "Ransomware",
```

```
    "SQL Injection",
```

```
    "Insider Threat",
```

```
    "XSS",
```

```
    "Botnet"
```

```
]
```

```
st.markdown(
```

```
    "<h1 style='text-align:center'>SMART THREAT DETECTION SYSTEM</h1>",
```

```
    unsafe_allow_html=True
```

```
)
```

```
st.markdown(
```

```
    "<h3 style='text-align:center;color:#9FB3C8'>AI-Powered Behavioral Analytics</h3>",
```

```
    unsafe_allow_html=True
```

```
)
```

```
# ===== ALERT CARDS =====
```

```
c1, c2, c3, c4 = st.columns(4)
```

```
def alert(col, label, value, color):
```

```
    col.markdown(f"""
```

```
    <div style='text-align:center; padding:10px;'>
```

```
        <div style='font-size:14px;color:#9FB3C8'>{label}</div>
```

```
<div style='font-size:34px;font-weight:800;color:{color}'>{value}</div>
</div>
""", unsafe_allow_html=True)
```

```
alert(c1, "Critical Threats", random.randint(10, 30), "#FF4C4C")
alert(c2, "High Risks", random.randint(20, 50), "#FF8C00")
alert(c3, "Medium Risks", random.randint(40, 90), "#FFD166")
alert(c4, "Low Risks", random.randint(80, 150), "#2ECC71")
```

```
# ===== MAIN GRAPHS =====
```

```
left, center, right = st.columns([1.3, 2.4, 1.7])
```

with left:

```
st.subheader("Risk Score Distribution")
df = pd.DataFrame({
    "Risk": ["Low", "Medium", "High", "Critical"],
    "Count": [random.randint(10, 80) for _ in range(4)]
})
fig_bar = px.bar(
    df, x="Risk", y="Count", color="Risk",
    color_discrete_sequence=["#2ECC71", "#FFD166", "#FF8C00", "#FF4C4C"],
    template="plotly_dark", height=300
)
st.plotly_chart(fig_bar, use_container_width=True, key=f"risk_bar_{t}")
```

with center:

```
st.subheader("Anomaly Trends (Live)")
df = pd.DataFrame({
    "Time": list(range(1, 13)),
    "Critical": [random.randint(20, 60) for _ in range(12)],
    "High": [random.randint(15, 50) for _ in range(12)],
```

```

    "Medium": [random.randint(10, 35) for _ in range(12)]
})

fig_line = px.line(
    df, x="Time", y=["Critical", "High", "Medium"],
    markers=True, template="plotly_dark", height=300
)

st.plotly_chart(fig_line, use_container_width=True, key=f"anomaly_line_{t}")

```

with right:

```

st.subheader("Real-Time Threat Map")

locs = {
    "India": (21, 78),
    "USA": (37, -95),
    "Germany": (51, 10),
    "UK": (55, -3),
    "Brazil": (-14, -51)
}

df_map = pd.DataFrame([
    [c, lat, lon, random.choice(ATTACK_TYPES)]
    for c, (lat, lon) in locs.items()
], columns=["Country", "Lat", "Lon", "Attack"])

fig_geo = px.scatter_geo(
    df_map,
    lat="Lat",
    lon="Lon",
    color="Attack",
    template="plotly_dark",
    height=300
)

```

```
st.plotly_chart(fig_geo, use_container_width=True, key=f"threat_map_{t}")
```

```
# ===== LOWER SECTION (RESTORED) =====
```

```
a, b, c = st.columns(3)
```

```
with a:
```

```
st.subheader("User Behavior Analysis")
```

```
st.write(f"Login Anomalies : *{random.randint(10, 40)}*")
```

```
st.write(f"Data Access Alerts : *{random.randint(5, 30)}*")
```

```
st.write(f"Unusual Activity : *{random.randint(20, 60)}*")
```

```
with b:
```

```
st.subheader("Threat Types Detected")
```

```
pie = pd.DataFrame({  
    "Attack": ATTACK_TYPES,  
    "Value": [random.randint(10, 40) for _ in ATTACK_TYPES]  
})
```

```
fig_pie = px.pie(  
    pie,  
    names="Attack",  
    values="Value",  
    hole=0.55,  
    template="plotly_dark",  
    height=300  
)
```

```
st.plotly_chart(fig_pie, use_container_width=True, key=f"threat_pie_{t}")
```

```
with c:
```

```
st.subheader("Recent Incidents")
```

```
for _ in range(3):
```

```
    st.write(random.choice([
```

```

"🔴 Ransomware Detected – CRITICAL",
"🟠 SQL Injection Attempt – HIGH",
"🟡 Insider Threat Activity – MEDIUM",
"🟠 XSS Attack Blocked – HIGH",
"🟡 Botnet Traffic Identified – MEDIUM"
)))

```

```
def ai_threat_detection_page():
```

```
    st.title("🛡️ AI Threat Detection System")
```

```
    uploaded = st.file_uploader("📁 Upload CSV", type=["csv"])
```

```
    if uploaded:
```

```
        try:
```

```
            df = pd.read_csv("final dataset.csv")
```

```
        except Exception as e:
```

```
            st.error(f"Failed to read uploaded CSV: {e}")
```

```
        return
```

```
    else:
```

```
        st.info("Please upload a CSV to run threat detection.")
```

```
        return
```

```
    st.subheader("📌 Uploaded Dataset Preview")
```

```
    st.dataframe(df.head())
```

```
# Map numeric Label -> Attack_Type if present
```

```
attack_map = {
```

```
    0: "Normal",
```

```
    1: "DoS Attack",
```

```
    2: "Unauthorized Access Attack",
```

```
    3: "Brute Force Attack",
```

```
4: "Malware Attack"
}
```

```
if "Label" in df.columns:
```

```
# try numeric mapping first, else fallback
```

```
try:
```

```
    df["Label_num"] = pd.to_numeric(df["Label"], errors="coerce").fillna(-1).astype(int)
```

```
    df["Attack_Type"] = df["Label_num"].map(attack_map).fillna(df.get("Label", "Unknown"))
```

```
    df.drop(columns=["Label_num"], inplace=True, errors="ignore")
```

```
except Exception:
```

```
    df["Attack_Type"] = df["Label"].map(lambda x: attack_map.get(x, str(x)))
```

```
else:
```

```
    if "Attack_Type" not in df.columns:
```

```
        df["Attack_Type"] = "Unknown"
```

```
if st.button("🚀 Run Threat Detection"):
```

```
    st.subheader("🔍 Full Threat Records")
```

```
    st.dataframe(df)
```

```
col1, col2, col3 = st.columns(3)
```

```
# ----- Vertical Bar Chart: Attack Types (colored) -----
```

```
with col1:
```

```
    attack_counts = df["Attack_Type"].value_counts().reset_index()
```

```
    attack_counts.columns = ["Attack_Type", "Count"]
```

```
fig1 = px.bar(
```

```
    attack_counts,
```

```
    x="Attack_Type",
```

```
    y="Count",
```

```
    color="Attack_Type",
```

```

    title="Attack Types (Vertical Bar Chart)",
    color_discrete_sequence=px.colors.qualitative.Bold
)

fig1.update_traces(texttemplate="%{y}", textposition="outside")
fig1.update_layout(xaxis_title="Attack Type", yaxis_title="Count", xaxis_tickangle=-45, height=420)

st.subheader("🇩🇪 Attack Types")

st.plotly_chart(fig1, use_container_width=True)

```

# ----- Location chart (pie) -----

with col2:

```

if "Location" in df.columns:

    loc_counts = df.groupby("Location")["Attack_Type"].count().reset_index()
    loc_counts.columns = ["Location", "Threats"]

    fig2 = px.pie(loc_counts, names="Location", values="Threats", hole=0.35, title="Threats by Location")

    st.subheader("📍 Threats by Location")

    st.plotly_chart(fig2, use_container_width=True)

```

else:

```

    st.info("No 'Location' column in dataset to show location chart.")

```

# ----- Line Chart: Device Threats (colored by device) -----

with col3:

```

if "Device" in df.columns:

    dev_counts = df.groupby("Device")["Attack_Type"].count().reset_index()
    dev_counts.columns = ["Device", "Threats"]

    # Sort devices alphabetically for consistent line order
    dev_counts_sorted = dev_counts.sort_values("Device")

```

```

fig3 = px.line(
    dev_counts_sorted,
    x="Device",
    y="Threats",

```

```

        markers=True,

        title="Device Threats (Line Chart)",

        color_discrete_sequence=px.colors.qualitative.Set2
    )

    fig3.update_traces(mode="lines+markers")

    fig3.update_layout(xaxis_title="Device", yaxis_title="Threat Count", height=420)

    st.subheader("📊 Device Threats")

    st.plotly_chart(fig3, use_container_width=True)

else:

    st.info("No 'Device' column in dataset to show device chart.")

```

```

# -----

# Remedies table (UNDER charts)

# -----

st.markdown("---")

st.subheader("💡 Recommended Remedies")

```

```

remedies_data = {
    "Attack_Type": [
        "Normal",
        "DDoS Attack",
        "Sql Injection",
        "Ransomware",
        "Insider Threat",
        "Zero-day Exploit",
        "Botnet"
    ],
    "Remedies": [
        "Maintain continuous monitoring, regular patching, and baseline behavior analysis to ensure system stability",
        "Mitigate DDoS attacks using traffic filtering, rate limiting, CDNs, and anomaly-based detection mechanisms",
    ]
}

```

```

        "Prevent SQL injection by using parameterized queries, input validation, and web application firewalls",
        "Defend against ransomware through regular offline backups, endpoint security solutions, and least-privilege access control",
        "Reduce insider threats by enforcing role-based access control, user behavior monitoring, and comprehensive audit logging",
        "Protect systems from zero-day exploits using behavior-based detection, sandboxing, and zero-trust security models",
        "Counter botnet attacks by monitoring command-and-control traffic, applying network segmentation, and patching vulnerable devices"
    ]
}

```

```

remedies_df = pd.DataFrame(remedies_data)
st.dataframe(remedies_df, use_container_width=True)

```

```

# ===== Threat Management Page =====

```

```

import streamlit as st
import pandas as pd
import os
import numpy as np
import matplotlib.pyplot as plt

```

```

def threat_management_page():
    st.title("Threat Management Dashboard")

```

```

# ----- Load Dataset -----

```

```

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
dataset_path = os.path.join(
    BASE_DIR, "..", "data", "FINAL_excel_safe_login_time_dataset_with_device_7500_added.csv"
)

```

```

try:

```

```

df = pd.read_csv("final dataset.csv")

except FileNotFoundError:

    st.error("Dataset not found. Place the CSV inside the data folder.")

    return


# ----- Normalize -----

df.columns = df.columns.str.lower()

df["attack_type"] = df["attack_type"].astype(str).str.lower()


# ----- Remove Unwanted Attacks -----

excluded_attacks = ["bruteforce", "phishing", "malware"]

df = df[~df["attack_type"].isin(excluded_attacks)]


# ----- Severity Mapping -----

severity_map = {

    "dos": "High",

    "ddos": "Critical",

    "sql injection": "High",

    "xss": "Medium",

    "insider threat": "High",

    "data exfiltration": "Critical"

}


df["severity"] = df["attack_type"].map(severity_map).fillna("Low")


# ----- AI Risk Score -----

risk_ranges = {

    "Low": (10, 30),

    "Medium": (40, 60),

    "High": (70, 85),

    "Critical": (90, 100)

```

```
}
```

```
df["ai_risk_score"] = df["severity"].apply(  
    lambda x: np.random.randint(*risk_ranges[x])  
)
```

```
df["status"] = "Open"
```

```
# ===== METRICS =====
```

```
col1, col2, col3 = st.columns(3)
```

```
col1.metric("Total Threats", len(df))
```

```
col2.metric("High / Critical Threats", len(df[df["severity"].isin(["High", "Critical"])]))
```

```
col3.metric("Average AI Risk Score", round(df["ai_risk_score"].mean(), 2))
```

```
st.markdown("---")
```

```
# ===== SIDE-BY-SIDE CHARTS =====
```

```
st.subheader("Threat Analysis Overview")
```

```
col_left, col_right = st.columns(2)
```

```
# ----- LEFT: Severity Distribution -----
```

```
with col_left:
```

```
    st.markdown("**Threat Severity Distribution**")
```

```
    severity_counts = df["severity"].value_counts()
```

```
    fig1, ax1 = plt.subplots(figsize=(3.2, 2.2))
```

```
    ax1.bar(  
        severity_counts.index,  
        severity_counts.values,  

```

```
color=["#4CAF50", "#FFC107", "#FF5722", "#9C27B0"]  
)
```

```
ax1.set_title("Severity Levels", fontsize=9, fontweight="bold")  
ax1.set_xlabel("Severity", fontsize=8)  
ax1.set_ylabel("Count", fontsize=8)  
ax1.tick_params(axis="both", labelsize=8)  
ax1.grid(axis="y", linestyle="--", alpha=0.4)
```

```
st.pyplot(fig1, use_container_width=True)
```

```
# ---- RIGHT: AI Risk Score ----
```

```
with col_right:
```

```
st.markdown("***AI Risk Score Distribution***")
```

```
fig2, ax2 = plt.subplots(figsize=(3.2, 2.2))
```

```
ax2.hist(  
    df["ai_risk_score"],  
    bins=10,  
    color="#03A9F4",  
    edgecolor="#0D47A1"  
)
```

```
ax2.set_title("AI Risk Scores", fontsize=9, fontweight="bold")  
ax2.set_xlabel("Risk Score", fontsize=8)  
ax2.set_ylabel("Frequency", fontsize=8)  
ax2.tick_params(axis="both", labelsize=8)  
ax2.grid(axis="y", linestyle="--", alpha=0.4)
```

```
st.pyplot(fig2, use_container_width=True)
```

```

st.markdown("---")

# ===== FILTER =====

st.subheader("Threat Records")

severity_filter = st.selectbox(
    "Filter by Severity Level",
    ["All", "Low", "Medium", "High", "Critical"]
)

filtered_df = df if severity_filter == "All" else df[df["severity"] == severity_filter]

# ===== TABLE =====

st.dataframe(
    filtered_df[["attack_type", "severity", "ai_risk_score", "status"]],
    use_container_width=True
)

def main_app():
    # ----- Sidebar -----

    st.sidebar.title("🌀 Navigation")

    st.sidebar.markdown(f"👤 **User:** {st.session_state.current_user}")

    page = st.sidebar.radio(
        "Go to",
        [
            "📊 Dashboard",
            "🤖 AI Threat Detection",
            "🛡️ Threat Management",
            "🚪 Logout"
        ]
    )

```

```
)
```

```
# ----- Page Routing -----
```

```
if page == "📊 Dashboard":
```

```
    dashboard_page()
```

```
elif page == "🤖 AI Threat Detection":
```

```
    ai_threat_detection_page()
```

```
elif page == "🛡️ Threat Management":
```

```
    threat_management_page()
```

```
elif page == "🚪 Logout":
```

```
    st.session_state.logged_in = False
```

```
    st.session_state.current_user = None
```

```
    st.success("Logged out successfully")
```

```
    time.sleep(1)
```

```
    st.rerun()
```

```
# ===== START =====
```

```
if st.session_state.logged_in:
```

```
    main_app()
```

```
else:
```

```
    if st.session_state.register_mode:
```

```
        register_page()
```

```
    else:
```

```
        login_page()
```